Proving Basic Algorithms Correct: Sorting

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Proving Basic Algorithms Correct: Sorting
Computer-Assisted Theorem Proving (7CCSACTP)
Mohammad Abdulaziz

Principle: Count function calls

For every function $f :: \tau_1 \Rightarrow \cdots \Rightarrow \tau_n \Rightarrow \tau$
define a *timing function* $t_f :: \tau_1 \Rightarrow \cdots \Rightarrow \tau_n \Rightarrow \text{nat}$:

Principle: Count function calls
For every function $f :: \tau_1 \Rightarrow \cdots \Rightarrow \tau_n \Rightarrow \tau$
define a timing function $t_f :: \tau_1 \Rightarrow \cdots \Rightarrow \tau_n \Rightarrow \text{nat}$:

- How do we formally prove running time in a functional setting?
- We define a timing function t_f for every defined function f.
- Translating defining equations: every function step costs + 1.
- Expressions sum the timing costs of subterms and function invocations.
- Primitive operations (constructors, booleans, numbers) cost 1.
- **Definition:** equations: $\dfrac{e \rightsquigarrow e'}{f\, p_1 \ldots p_n = e \;\rightsquigarrow\; t_f\, p_1 \ldots p_n = e' + 1}$; expressions: $\dfrac{s_1 \rightsquigarrow t_1 \quad \ldots \quad s_k \rightsquigarrow t_k}{g\, s_1 \ldots s_k \rightsquigarrow t_1 + \cdots + t_k + t_g\, s_1 \ldots s_k}$; variables, constants, constructors, primitive operations on `bool` and numbers cost 1.

Example

```
app [] ys = ys
```

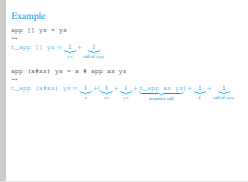
$\rightsquigarrow$

$$\texttt{t_app [] ys} = \underbrace{1}_{\texttt{ys}} + \underbrace{1}_{\text{call of } \texttt{app}}$$

```
app (x#xs) ys = x # app xs ys
```

$\rightsquigarrow$

$$\texttt{t_app (x\#xs) ys} = \underbrace{1}_{\texttt{x}} + (\underbrace{1}_{\texttt{xs}} + \underbrace{1}_{\texttt{ys}} + \underbrace{\texttt{t_app xs ys}}_{\text{recursive call}}) + \underbrace{1}_{\texttt{\#}} + \underbrace{1}_{\text{call of } \texttt{app}}$$



- Translate each defining equation of `app`: the cost of evaluating its right-hand side, plus 1 for the call of `app` itself.
- `app [] ys = ys`: the right-hand side is the variable `ys`, which costs 1. Plus 1 for the call: `t_app [] ys = 1 + 1`.
- `app (x#xs) ys = x # app xs ys`: the right-hand side is the constructor `#` applied to `x` and `app xs ys`.
  - `x`: a variable, costs 1.
  - `app xs ys`: a call of a defined function, so its arguments `xs` and `ys` (1 each) plus the time of the call, `t_app xs ys`.
  - `#`: a constructor, costs 1.
  - Plus 1 for the call of `app (x#xs) ys` itself.
- Only `t_app xs ys` depends on the input; everything else is a constant. This is why the next slides can reduce the constants to 1.

A compact formulation of $e \rightsquigarrow t$

t is the sum of all $t_g\ s_1 \ldots s_k$
such that $g\ s_1 \ldots s_k$ is a subterm of e

If g is

- a variable, a constant, a constructor or
- a predefined function on `bool` or numbers

then $t_g \cdots = 1$.

- Compact rule: t is simply the sum of all t_g s_1 ... s_k for every subterm g s_1 ... s_k in e.
- If g is a primitive, variable, or constructor, its timing cost is 1.

if and *case*

So far we model a call-by-value semantics

Conditionals and case expressions are evaluated lazily.

Translation:

$$\frac{b \rightsquigarrow t \quad s_1 \rightsquigarrow t_1 \quad s_2 \rightsquigarrow t_2}{\textbf{if } b \textbf{ then } s_1 \textbf{ else } s_2 \rightsquigarrow t + (\textbf{if } b \textbf{ then } t_1 \textbf{ else } t_2)}$$

Similarly for **case**

- So far we modelled eager/call-by-value evaluation.
- Conditionals and case expressions, however, are evaluated lazily!
- We only pay for the condition b, plus the branch that is actually executed.

$O(\cdot)$ is enough

$\Longrightarrow$ Reduce all additive constants to 1

Example

```
t_app (x#xs) ys = t_app xs ys + 1
```

$\Longrightarrow$ Count only

- the defined functions via t_f and
- 1 for the function call.

All other operations (variables etc) cost 0, not 1.

- For asymptotic O(.) bounds, precise constants do not matter.
- We reduce all additive constants to 1.
- We count only defined function calls via t_f, and 1 for each function call invocation.
- All other operations (variable lookups, constructors) cost 0.

Discussion

- The definition of t_f from f can be automated.
- The correctness of t_f could be proved w.r.t.
  a semantics that counts computation steps.
- Precise complexity bounds (as opposed to $O(\cdot)$) would require a formal model of (at least) the compiler and the hardware.

Discussion
- The definition of t_f from f can be automated.
- The correctness of t_f could be proved w.r.t.
  a semantics that counts computation steps.
- Precise complexity bounds (as opposed to $O(\cdot)$) would require a formal model of (at least) the compiler and the hardware.

- Automated synthesis of t_f from f is possible and supported in modern tooling.
- Proving correctness of t_f requires a operational step semantics.
- Exact clock-cycle or memory bounds would require formal models of compilers and hardware architectures.